

LAPORAN TUGAS BESAR

IF1221 Logika Komputasional

AturAturAja



Dipersiapkan oleh:

- | | |
|------------------------------|----------|
| 1. Mochamad Fachri Alfaridzi | 13525128 |
| 2. Haikal Muhammad Royyan | 13525021 |
| 3. Moch Naufal Zaki Basara | 13525095 |
| 4. Zidane Uland Fakhry | 13525132 |

Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

	Sekolah Teknik Elektro dan Informatika ITB	Nomor Dokumen		Halaman
		<i>IF1221-G05</i>		<i>49</i>
		<i>Revisi</i>	<i>2</i>	<i>28 Mei 2026</i>

Daftar Isi

1	Ringkasan.....	3
2	Dynamic Predicate.....	4
3	Fakta.....	5
	3.1 Representasi Kartu.....	5
4	Rule.....	6
	4.1 Gameplay.....	6
	4.1.1 clearGame.....	6
	4.1.2 nextTurn.....	7
	4.1.3 nextPlayer.....	7
	4.1.4 cekAdaKartu.....	8
	4.1.5 mainkanKartu.....	8
	4.1.6 lanjut, cekAdaExit, cekHasil.....	9
	4.1.7 ambilKartu.....	9
	4.1.8 jalankanEfek.....	10
	4.1.8.1 skip.....	10
	4.1.8.2 reverse.....	10
	4.1.8.3 drawTwo.....	10
	4.1.8.4 wild.....	10
	4.1.8.5 wild draw four.....	11
	4.1.8.6 mimic.....	11
	4.1.9 tantang.....	11
	4.1.10 uni.....	11
	4.1.11 tangkap.....	12
	4.2 Setup.....	12
	4.2.1 startGame.....	12
	4.2.2 bacaJumlahPemain.....	13
	4.2.3 inputPemain.....	14
	a. cekNama.....	14
	4.2.4 acakList.....	15
	a. ambilElemenKe.....	15
	4.3 Info.....	16
	4.3.1 lihatKartu.....	16
	4.3.2 cekInfo.....	17
	4.3.3 Perhitungan Poin dan Hasil Akhir.....	18
	4.4 Save_load.....	19

4.4.1 saveGame.....	20
4.4.2 loadGame.....	22
5 Alur Permainan.....	2
5.1 Awal Permainan.....	23
5.2 Support Command.....	25
5.3 Validitas Kartu.....	27
5.4 Kartu Aksi.....	29
5.5 UNI dan Tangkap.....	32
5.6 End Game.....	34
5.7 Save load.....	36
5.8 Contoh Full Game.....	38
6 Pembagian Kerja dalam Kelompok.....	45

1 Ringkasan

Tugas besar ini mengimplementasikan simulasi permainan kartu berbasis terminal menggunakan bahasa Prolog. Program mensimulasikan mekanisme permainan seperti giliran pemain, pengambilan kartu, validasi kartu, serta berbagai efek kartu seperti *skip*, *reverse*, *draw two*, *wild*, dan *wild draw four*.

Selain itu, sistem juga mendukung fitur tambahan seperti tantangan, mekanisme “UNI” ketika pemain memiliki sisa satu kartu, serta sistem penalti jika pemain melanggar aturan tertentu.

Implementasi dilakukan dengan menggunakan pendekatan deklaratif Prolog yang memanfaatkan fakta dan aturan, serta *dynamic predicate* untuk merepresentasikan status permainan yang berubah selama program berjalan.

Laporan ini mencakup penjelasan struktur program, implementasi fitur utama dan tambahan, serta mekanisme pengelolaan status permainan. Seluruh fitur utama berhasil diimplementasikan dan program dapat berjalan sesuai spesifikasi yang ditentukan.

2 Dynamic Predicate

Fakta	Argumen	Keterangan
pemain/1	(ListPemain)	Menyimpan daftar pemain yang telah diacak ketika game dimulai.
giliran/1	(NamaPemain)	Menyimpan data pemain mana yang sedang dapat giliran.
kartuPemain/2	(NamaPemain, ListKartu)	Menyimpan daftar kartu yang dipegang masing-masing pemain dalam bentuk list kartu(Warna, Jenis).
discardTop/1	(kartu(Warna, Jenis))	Menyimpan kartu di atas tumpukan kartu. Kartu yang terakhir dimainkan.
warnaAktif/1	(Warna)	Warna kartu yang valid sekarang. Diperbarui setiap kartu Wild atau Mimic digunakan.
warnaLama/1	(Warna)	Menyimpan warna aktif sebelum kartu Wild Draw Four digunakan atau ketika mekanisme tantangan.

arahPermainan/1	(kanan kiri)	Menyimpan arah giliran saat ini. Dibalik ketika kartu reverse dimainkan.
statusUNI/1	(ListPemain)	Menyimpan daftar pemain yang telah sah mengucapkan “UNI” (uni(2) dalam game). Digunakan untuk mencegah salah tangkap pada mekanisme tangkap.
deckAktif/1	(ListKartu)	Menyimpan sisa kartu dalam deck yang belum dibagikan. Pemain yang menggunakan fitur ambilKartu atau kena penalti akan mengambil dari sini.
riwayatAksi/1	(ListAksi)	Menyimpan riwayat kartu aksi yang telah dimainkan.
efekPending/1	(Efek tidak_ada)	Menyimpan efek “yang sedang menunggu” yang memengaruhi pilihan aksi pemain berikutnya.

3 Fakta

Dalam program ini, fakta digunakan untuk merepresentasikan struktur dasar permainan kartu yang bersifat tetap pada awal inisialisasi sistem. Fakta ini tidak berubah secara langsung selama eksekusi program, kecuali melalui mekanisme pembaruan status menggunakan *retract* dan *assertz* pada bagian *dynamic predicate*.

Fakta berperan sebagai dasar logika permainan, terutama dalam mendefinisikan inti seperti representasi kartu, struktur pemain, serta kondisi awal permainan sebelum interaksi terjadi.

3.1 Representasi Kartu

Fakta utama dalam sistem ini adalah representasi kartu menggunakan struktur:

```
kartu(Warna, Tipe)

/* Contoh instansiasi fakta kartu: */
kartu(merah, 5).
kartu(biru, reverse).
kartu(hitam, wild).
kartu(hitam, wild_draw_four).
```

Fakta ini menjadi dasar dalam validasi kartu, penyimpanan kartu pemain, dan kartu yang sedang aktif.

3.2 Struktur Awal Pemain

Fakta ini digunakan untuk mendefinisikan urutan awal pemain dalam permainan:

```
/* Contoh: */  
pemain([p1, p2, p3, p4]).
```

Fakta ini berfungsi untuk menentukan urutan giliran awal, dasar rotasi pemain, dan mendukung fitur reverse arah permainan.

3.3 Kondisi Arah Permainan Awal

Fakta ini digunakan untuk menyatakan arah awal permainan:

```
/* Contoh: */  
arahPermainan(kanan).
```

Fakta ini menentukan arah giliran *default* dan dasar perubahan arah melalui kartu *reverse*.

3.4 Kondisi Warna Awal

Pada awal permainan, warna aktif dapat didefinisikan sebagai fakta:

```
/* Contoh: */  
warnaAktif(merah).
```

Fakta ini digunakan untuk validasi kesesuaian kartu dengan warna yang aktif dan perubahan warna pada kartu *wild*.

4 Rule

4.1 Gameplay

4.1.1 clearGame

Predikat `clearGame` berfungsi untuk mereset seluruh state game dengan menghapus semua fakta dinamis yang tersimpan dalam database prolog. Predikat ini dipanggil setelah permainan selesai untuk membersihkan memori.

Implementasi menggunakan **`retractall/1`** untuk menghapus semua fakta dengan funktor tertentu tanpa melempar error meskipun fakta tidak ditemukan.

```
clearGame :-  
    retractall(pemain(_)),  
    retractall(giliran(_)),  
    retractall(discardTop(_)),  
    retractall(warnaAktif(_)),  
    retractall(arahPermainan(_)),  
    retractall(statusUNI(_)),  
    retractall(kartuPemain(_, _)),  
    retractall(deckAktif(_)).
```

Fakta-fakta yang dihapus meliputi: daftar pemain, giliran aktif, kartu teratas discard pile, warna aktif, arah permainan, status UNI, kartu tangan setiap pemain, dan deck aktif.

4.1.2 nextTurn

Predikat **nextTurn** berfungsi untuk memindahkan giliran ke pemain berikutnya sesuai arah permainan yang sedang berlaku (kanan atau kiri). Predikat ini dipanggil setiap kali giliran perlu berpindah.

```
nextTurn :-
    arahPermainan(kanan), !,
    pemain(List),
    giliran(Sekarang),
    nextPlayer(List, Sekarang, Berikutnya),
    retract(giliran(Sekarang)),
    assertz(giliran(Berikutnya)).

nextTurn :-
    arahPermainan(kiri),
    pemain(List),
    giliran(Sekarang),
    prevPlayer(List, Sekarang, Berikutnya),
    retract(giliran(Sekarang)),
    assertz(giliran(Berikutnya)).
```

Klausul pertama menangani arah **kanan** (urutan maju) dengan menggunakan **nextPlayer**, sedangkan klausul kedua menangani arah **kiri** (urutan mundur) menggunakan **prevPlayer**. Cut (!) pada klausul pertama mencegah backtracking ke klausul kedua.

4.1.3 nextPlayer

Predikat **nextPlayer/3** menentukan pemain berikutnya dari sebuah list pemain secara siklis. Jika pemain terakhir dalam list, maka pemain berikutnya adalah pemain pertama (melingkar).

```
nextPlayer([X,Y|_], X, Y).
nextPlayer([_|Tail], X, Y) :-
    nextPlayer(Tail, X, Y).
nextPlayer([Last], Last, First) :-
    pemain([First|_]).
```

Klausul pertama menangani kasus di mana pemain saat ini adalah elemen pertama list, mengembalikan elemen kedua sebagai berikutnya. Klausul kedua melakukan rekursi menelusuri list. Klausul ketiga menangani wrap-around: jika pemain saat ini adalah elemen terakhir, maka berikutnya adalah elemen pertama dari fakta **pemain/1**.

4.1.4 cekAdaKartu

Predikat **cekAdaKartu/2** memeriksa apakah ada terdapat apa ada setidaknya satu list kartu valid terhadap kartu referensi dalam list kartu. Digunakan apakah pemain punya kartu yang bisa dimainkan.

```
cekAdaKartu([],_) :- fail.
cekAdaKartu([H|_],X) :-
    kartuValid(H,X), !.
cekAdaKartu([_|T],X) :-
    cekAdaKartu(T,X).
```

Klausul pertama menangani list kosong dengan gagal (**fail**). Klausul kedua memeriksa head list menggunakan **kartuValid/2**; jika valid, predikat berhasil dengan cut untuk mencegah backtracking. Klausul ketiga melakukan rekursi pada tail jika head tidak valid.

4.1.5 mainkanKartu

Predikat **mainkanKartu/1** merupakan predikat utama untuk memainkan kartu dari tangan pemain. Menerima parameter index kartu dalam list tangan pemain.

```
mainkanKartu(Index) :-
    giliran(Pemain),
    kartuPemain(Pemain, ListKartu),
    ambilKartuKe(Index, ListKartu, KartuDipilih, SisaKartu),
    discardTop(KartuAtas),
    kartuValid(KartuDipilih, KartuAtas),
    retract(kartuPemain(Pemain, ListKartu)),
    assertz(kartuPemain(Pemain, SisaKartu)),
    ...
    jalankanEfek(KartuDipilih),
    lanjut.
```

Alur kerja predikat ini: mengambil kartu pada index tertentu, memvalidasi kartu terhadap kartu atas discard pile, memperbarui tangan pemain, memeriksa kondisi UNI, memperbarui discard pile dan warna aktif, menampilkan informasi, lalu menjalankan efek kartu dan melanjutkan ke tahap berikutnya.

4.1.6 lanjut, cekAdaExit, cekHasil

Ketiga predikat ini bekerja bersama untuk menentukan apakah permainan telah berakhir atau perlu dilanjutkan.


```

lanjut :- cekAdaExit, !.
lanjut :- nextTurn.

cekAdaExit :-
    giliran(X),
    kartuPemain(X, ListKartu),
    length(ListKartu, 0), !,
    write('game selesai'), nl,
    cekHasil,
    clearGame.

cekHasil :-
    sumAllPlayer(X),
    pemain(Y),
    sort_with_id(X, Y, R),
    printList(R).

```

lanjut mencoba **cekAdaExit** terlebih dahulu; jika permainan belum selesai, maka memanggil **nextTurn**. Predikat **cekAdaExit** memeriksa apakah pemain yang baru saja bermain memiliki 0 kartu tersisa. Jika ya, permainan selesai dan **cekHasil** dipanggil untuk menampilkan urutan pemenang berdasarkan skor, lalu state di-reset.

4.1.7 ambilKartu

Predikat **ambilKartu** memungkinkan pemain mengambil satu kartu dari deck aktif ke tangan mereka, lalu memindahkan giliran ke pemain berikutnya.

```

ambilKartu :-
    giliran(P),
    deckAktif([KartuBaru|SisaDeck]),
    retract(deckAktif(_)),
    assertz(deckAktif(SisaDeck)),
    kartuPemain(P, ListLama),
    retract(kartuPemain(P, _)),
    assertz(kartuPemain(P, [KartuBaru|ListLama])),
    hapusUNI(P),
    write('Kamu ngambil kartu dari deck kartu!'), nl,
    nextTurn.

```

Kartu baru diambil dari head **deckAktif** dan dimasukkan ke head list tangan pemain. Jika pemain sebelumnya telah berteriak UNI, maka status UNI-nya dihapus karena jumlah kartu bertambah. Setelah itu giliran berpindah.

4.1.8 jalankanEfek

Predikat **jalankanEfek/1** menangani kasus khusus dari setiap kartu aksi yang ada. Terdapat lima jenis kartu aksi.

4.1.8.1 skip

Kartu skip menyebabkan pemain berikutnya kehilangan gilirannya. Predikat langsung memanggil **nextPlayer** untuk melewati giliran pemain yang ditarget.

```
jalankanEfek(kartu(_, skip)) :- !,  
    pemain(List), giliran(Sekarang),  
    nextPlayer(List, Sekarang, Berikutnya),  
    retract(giliran(Sekarang)),  
    assertz(giliran(Berikutnya)), ...
```

4.1.8.2 reverse

Kartu reverse membalik arah permainan. Memanggil helper **ubahArah** yang menukar nilai fakta **arahPermainan** antara **kanan** dan **kiri**.

```
jalankanEfek(kartu(_, reverse)) :- !,  
    ubahArah, ...
```

4.1.8.3 drawTwo

Kartu draw_two membuat pemain berikutnya mengambil 2 kartu dan kehilangan giliran.

```
jalankanEfek(kartu(_, draw_two)) :- !,  
    pemain(List), giliran(Sekarang),  
    nextPlayer(List, Sekarang, Target),  
    tambahKartu(Target, 2),  
    retract(giliran(Sekarang)),  
    assertz(giliran(Target)), ...
```

4.1.8.4 Wild

Kartu wild memungkinkan pemain memilih warna aktif baru. Pemain diminta memasukkan warna pilihan, kemudian **warnaAktif** diperbarui.

```
jalankanEfek(kartu(hitam, wild)) :- !,  
    write('Pilih warna baru(merah, kuning, hijau, biru): '),  
    read(WarnaBaru),  
    retract(warnaAktif(_)),  
    assertz(warnaAktif(WarnaBaru)), ...
```

4.1.8.5 Wild draw four

Kartu wild_draw_four menggabungkan efek wild (pilih warna) dan draw_two (target mendapat 4 kartu dan kehilangan giliran). Selain itu, warna sebelumnya disimpan dalam **warnaLama** untuk keperluan fiturantang.

```
jalankanEfek(kartu(hitam, wild_draw_four)) :- !,  
    warnaAktif(WarnaSebelumnya),  
    assertz(warnaLama(WarnaSebelumnya)),  
    read(WarnaBaru), ...,  
    tambahKartu(Target, 4), ...
```

4.1.8.6 Mimic

Kartu mimic berfungsi meniru kartu aksi yang telah dimainkan sebelumnya. Jika tidak ada kartu aksi yang telah dimainkan, maka mimic akan berfungsi sama persis seperti kartu wild.

```
jalankanEfek(kartu(hitam,mimic)) :- !,  
    write('Menelusuri riwayat permainan'), nl.  
    cariAksiTerakhir(HasilCari),  
    jalankanMimic(HasilCari),
```

Kartu mimic dibantu helper `jalankanEfekMimic` serta `cariAksiTerakhir`.

4.1.9 tantang

Predikat **tantang** memungkinkan pemain menantang keabsahan penggunaan kartu **wild_draw_four** oleh pemain sebelumnya. Tantangan berhasil jika pemain yang memainkan **wild_draw_four** sebenarnya memiliki kartu berwarna yang sama dengan warna sebelum kartu wild dimainkan.

```
tantang :-  
    giliran(Penantang),  
    pemain(List).
```

4.1.10 uni

Predikat **uni/1** digunakan pemain untuk menyebutkan 'UNI' saat memainkan kartu terakhir keduanya. Pemain harus memanggilnya bersamaan dengan memainkan kartu (melalui parameter `index`).

```
uni(Index) :-  
    giliran(P),  
    kartuPemain(P, List),  
    len(List, 2), !,  
    mainkanKartu(Index),  
    retract(statusUNI(L)),  
    assertz(statusUNI([P|L])), ...  
  
uni(_) :-  
    giliran(P),  
    tambahKartu(P, 1),  
    nextTurn.
```

Klausul pertama berhasil hanya jika pemain memiliki tepat 2 kartu sebelum memainkan. Setelah kartu dimainkan, pemain ditambahkan ke **statusUNI**. Klausul kedua adalah penalti: jika kondisi tidak terpenuhi, pemain mendapat 1 kartu tambahan.

4.1.11 tangkap

Predikat **tangkap/1** memungkinkan pemain menangkap pemain lain yang lupa menyebutkan UNI saat tersisa 1 kartu. Menerima nama target sebagai parameter.

```
tangkap(Target) :-
```

```

    kartuPemain(Target, L),
    len(L, 1),
    statusUNI(StatusList),
    \+ ada(Target, StatusList), !,
    tambahKartu(Target, 2), ...

tangkap(_) :-
    giliran(Penangkap),
    tambahKartu(Penangkap, 1),
    nextTurn.

```

Klausul pertama memeriksa target memiliki 1 kartu dan target **tidak** terdaftar dalam **statusUNI**. Jika terpenuhi, target mendapat penalti 2 kartu. Jika salah tangkap (target sudah UNI atau kartunya bukan 1), penangkap mendapat penalti 1 kartu.

4.2 Setup

4.2.1 startGame

```

startGame :-
    clearGame,
    bacaJumlahPemain(Jumlah),
    inputPemain(Jumlah, [], ListInput),
    acakList(ListInput, ListPemain),
    assertz(pemain(ListPemain)),
    ListPemain = [First|_],
    assertz(giliran(First)),
    assertz(arahPermainan(kanan)),
    assertz(statusUNI([])),
    deck(DeckAwal),
    bagiSemua(ListPemain, DeckAwal, DeckSisa),
    initDiscard(DeckSisa, DeckAkhir),
    assertz(deckAktif(DeckAkhir)),
    write('Game berhasil dimulai. '), nl,
    pemain(X), write('Urutan paermainanya adalah: '), write(X), nl,
    discardTop(Y), write('Kartu Top pertama: '), write(Y), nl,
    giliran(Z), write('Giliran '), write(Z), nl, !.

```

`startGame` menjadi titik awal permainan UNI. Perintah ini membersihkan state permainan sebelumnya, membaca jumlah pemain, menerima nama pemain, mengacak urutan giliran, membagikan kartu awal, menentukan kartu pertama pada discard pile, lalu menetapkan pemain

yang mendapat giliran pertama. Perintah ini berjalan valid jika jumlah pemain berada pada rentang 2 sampai 4 dan seluruh nama pemain unik; jika tidak, program akan meminta input ulang sesuai bagian yang tidak valid.

4.2.2 bacaJumlahPemain

```
bacaJumlahPemain(Jumlah) :-  
    write('Masukkan jumlah pemain: '),  
    read(Input),  
    validJumlahPemain(Input), !,  
    Jumlah = Input.  
  
bacaJumlahPemain(Jumlah) :-  
    write('Mohon masukkan angka antara 2 - 4. '), nl,  
    bacaJumlahPemain(Jumlah).  
/* Helper*/  
validJumlahPemain(Jumlah) :-  
    Jumlah >= 2,  
    Jumlah <= 4.
```

`bacaJumlahPemain` berfungsi untuk menerima input jumlah pemain dari pengguna. Jumlah pemain hanya diterima jika berada pada rentang 2 sampai 4. Jika pengguna memasukkan angka di luar rentang tersebut, program menampilkan pesan kesalahan dan meminta input kembali sampai jumlah pemain valid diperoleh.

4.2.3 inputPemain

```
inputPemain(0, _, []).  
inputPemain(N, SudahAda, [Nama1|Tail]) :-  
    N > 0,  
    write('Masukkan nama pemain: '),  
    read(Nama),  
    cekNama(Nama, SudahAda, Nama1),  
  
    N1 is N - 1,  
    inputPemain(  
        N1,  
        [Nama1|SudahAda],  
        Tail
```

```
) .
```

inputPemain digunakan untuk membaca nama pemain sebanyak jumlah pemain yang sudah ditentukan. Setiap nama yang dimasukkan akan diperiksa agar tidak sama dengan nama pemain sebelumnya. Jika nama valid, nama tersebut disimpan ke dalam daftar pemain. jika nama sudah digunakan, program meminta pengguna memasukkan nama lain.

a. cekNama

```
cekNama (Nama, List, Result) :-  
    anggotaList (Nama, List),  
    write('Nama Sudah Digunakan. Masukkan nama lain: '),  
    read (Nama1),  
    cekNama (Nama1, List, Result).  
  
cekNama (Nama, List, Nama) :-  
    \+ anggotaList (Nama, List).  
  
/* Helper*/  
anggotaList (X, [X|_]).  
anggotaList (X, [_|Tail]) :-  
    anggotaList (X, Tail).
```

cekNama menangani validasi nama pemain ketika proses input berlangsung. Rule ini mengecek apakah nama yang dimasukkan sudah terdapat pada daftar nama sebelumnya. Jika nama sudah ada, pengguna diminta memasukkan nama pengganti. jika belum ada, nama tersebut langsung diterima sebagai nama pemain. Rule anggotaList dipakai sebagai helper ketika program perlu mengetahui apakah suatu nama sudah ada di dalam list pemain atau belum.

4.2.4 acakList

```
acakList([], []).  
acakList(List, [Pilihan|TailAcak]) :-  
    panjangList(List, Panjang),  
    BatasAtas is Panjang + 1,  
    random(1, BatasAtas, Index),  
    ambilElemenKe(Index, List, Pilihan, Sisa),  
    acakList(Sisa, TailAcak).  
  
/* Helper */  
panjangList([], 0).  
panjangList([_|Tail], Panjang) :-  
    panjangList(Tail, PanjangTail),
```

```
Panjang is PanjangTail + 1.
```

acakList digunakan untuk mengacak daftar pemain sebelum permainan dimulai. Rule ini memilih satu elemen secara acak dari list pemain, memasukkannya ke list hasil, lalu melanjutkan proses terhadap sisa list secara rekursif. Proses berhenti ketika list asal sudah kosong, sehingga seluruh pemain masuk ke urutan baru yang sudah teracak. Rule panjangList dipakai sebagai helper ketika program perlu panjang suatu list.

a. ambilElemenKe

```
ambilElemenKe(1, [H|T], H, T) :- !.  
ambilElemenKe(N, [H|T], Pilihan, [H|Sisa]) :-  
    N > 1,  
    N1 is N - 1,  
    ambilElemenKe(N1, T, Pilihan, Sisa).
```

ambilElemenKe digunakan untuk mengambil elemen pada posisi tertentu dari sebuah list. Jika indeks yang dicari adalah 1, elemen pertama langsung diambil; jika indeks lebih besar, pencarian dilanjutkan ke tail sambil mempertahankan elemen lain sebagai bagian dari sisa list. Rule ini membantu proses pengacakan karena elemen yang sudah dipilih dapat dipisahkan dari list asal.

4.3 Info

4.3.1 lihatKartu

Predikat lihatKartu menampilkan seluruh kartu yang dimiliki pemain yang sedang mendapat giliran. Setiap kartu ditampilkan dengan nomor urut dalam format Warna-Jenis.

```
lihatKartu :-  
    giliran(Pemain),  
    kartuPemain(Pemain, ListKartu),  
    write('Berikut kartu yang anda miliki. '), nl,  
    tampilkanKartu(ListKartu, 1).  
  
tampilkanKartu([], _).  
tampilkanKartu([kartu(Warna, Jenis) | Tail], Nomor) :-  
    write(Nomor),  
    write(' '),  
    write(Warna),  
    write('-'),  
    write(Jenis),
```

```
nl,  
Next is Nomor + 1,  
tampilkanKartu(Tail, Next).
```

Predikat ini mengambil giliran aktif melalui giliran/1, lalu kartu tangan pemain tersebut melalui kartuPemain/2. Helper tampilkanKartu/2 melakukan rekursi terhadap list kartu sambil mencetak nomor urut yang bertambah pada setiap langkah.

4.3.2 cekInfo

Predikat cekInfo menampilkan ringkasan kondisi permainan saat ini: kartu teratas discard pile, urutan pemain, arah permainan, giliran aktif, serta nama dan jumlah kartu setiap pemain.

```
cekInfo :-  
    discardTop(kartu(Warna, Jenis)),  
    write('Kartu discard top: '),  
    write(Warna),  
    write('-'),  
    write(Jenis),  
    nl,  
    pemain(ListPemain),  
    write('Urutan pemain: '),  
    write(ListPemain), nl,  
    arahPermainan(Arah),  
    write('Arah permainan: '), write(Arah), nl,  
    giliran(A), write('Giliran: '), write(A),  
    nl, nl,  
    tampilkanInfoPemain(ListPemain, 1).  
  
tampilkanInfoPemain([], _).  
tampilkanInfoPemain([P|Tail], Nomor) :-  
    kartuPemain(P, ListKartu),  
    len(ListKartu, Jumlah),  
    write('Nama pemain '),  
    write(Nomor),  
    write(': '),  
    write(P),  
    nl,  
    write('Jumlah kartu: '),  
    write(Jumlah),  
    nl, nl,  
    Next is Nomor + 1,
```



```
tampilkanInfoPemain(Tail, Next).
```

Predikat ini menampilkan informasi global permainan (discard top, urutan pemain, arah, dan giliran), kemudian memanggil `tampilkanInfoPemain/2` untuk mencetak nama dan jumlah kartu tiap pemain secara rekursif. Helper `len/2` digunakan untuk menghitung panjang list kartu setiap pemain.

```
len([], 0).  
len([_|X], Y) :-  
    len(X, Y1),  
    Y is Y1 + 1.
```

4.3.3 Perhitungan Poin dan Hasil Akhir

Selain menampilkan informasi, berkas ini juga memuat predikat untuk menghitung poin sisa kartu dan menentukan peringkat pemain di akhir permainan. Poin setiap kartu dihitung oleh `sumListKartu/2`, dengan ketentuan: kartu angka bernilai sesuai angkanya (0–9), kartu aksi (`skip`, `reverse`, `draw_two`) bernilai 10, serta kartu hitam (`wild`, `wild_draw_four`, `mimic`) bernilai 20.

```
sumListKartu([], 0).  
sumListKartu([kartu(_,0)|Tail], X) :-  
    sumListKartu(Tail, X1),  
    X is X1 + 0.  
/* ... pola serupa untuk angka 1 sampai 9 ... */  
sumListKartu([kartu(_,skip)|Tail], X) :-  
    sumListKartu(Tail, X1),  
    X is X1 + 10.  
sumListKartu([kartu(_,wild)|Tail], X) :-  
    sumListKartu(Tail, X1),  
    X is X1 + 20.  
/* ... dan seterusnya ... */
```

Hasil akhir ditentukan oleh `cekAdaExit` yang dipanggil ketika seorang pemain kehabisan kartu. Predikat ini mencetak pesan akhir, menghitung poin sisa kartu seluruh pemain melalui `printSemuaPoin/1`, lalu menampilkan urutan pemenang menggunakan `sort_with_id/3` (sorting berbasis insertion sort yang mengurutkan pemain dari poin terkecil ke terbesar).

```
cekAdaExit :-  
    giliran(X),  
    kartuPemain(X, ListKartu),
```

```

ListKartu = [], !,
nl,
write('game selesai'), nl,
write('Berikut perhitungan poin sisa kartu.'), nl,
pemain(ListPemain),
printSemuaPoin(ListPemain),
nl,
write('urutan pemain: '), nl,
cekHasil,
clearGame.

X is X1 + 10.
sumListKartu([kartu(_,draw_two)|Tail],X):-
    sumListKartu(Tail,X1),
    X is X1 + 10.
sumListKartu([kartu(_,wild)|Tail],X):-
    sumListKartu(Tail,X1),
    X is X1 + 20.
sumListKartu([kartu(_,wild_draw_four)|Tail],X):-
    sumListKartu(Tail,X1),
    X is X1 + 20.
sumListKartu([kartu(_,mimic)|Tail],X):-
    sumListKartu(Tail,X1),
    X is X1 + 20.

```

4.4 Save_load

4.4.1 saveGame

Predikat saveGame menyimpan seluruh kondisi permainan saat ini ke dalam sebuah berkas .txt. Program meminta pemain memasukkan nama berkas, lalu menambahkan ekstensi .txt secara otomatis. Jika berkas dengan nama tersebut sudah ada, isinya akan ditimpa.

```

saveGame :-
    write('Masukkan nama file penyimpanan: '),
    read>NamaFile),
    atom_concat>NamaFile, '.txt',>NamaLengkap),
    open>NamaLengkap, write, Stream),
    tulisState(Stream),
    close(Stream),
    write('Status permainan berhasil disimpan ke '),
    write>NamaLengkap),
    write('.'). nl.

```

Alur kerjanya: membaca nama berkas dari pengguna dengan read/1, menggabungkannya dengan ekstensi .txt melalui atom_concat/3, membuka berkas mode write menggunakan open/3, menuliskan seluruh state melalui tulisState/1, lalu menutup berkas dengan close/1. Mode write pada open/3 otomatis menimpa isi berkas lama bila sudah ada.

Predikat tulisState/1 menuliskan setiap fakta dinamis ke stream dengan format key:value per baris.

```
tulisState(Stream) :-
    pemain(ListPemain),
    tulisBaris(Stream, 'urutan_pemain', ListPemain),
    giliran(P),
    tulisBaris(Stream, 'giliran', P),
    discardTop(kartu(W,J)),
    tulisBaris(Stream, 'discard_top', W-J),
    tulisSemuaKartu(Stream, ListPemain),
    arahPermainan(Arah),
    tulisBaris(Stream, 'arah_permainan', Arah),
    warnaAktif(WA),
    tulisBaris(Stream, 'warna_aktif', WA),
    statusUNI(SU),
    tulisBaris(Stream, 'status_UNI', SU),
    deckAktif(Deck),
    tulisBaris(Stream, 'deck_aktif', Deck),
    ( riwayatAksi([PemainAksi-kartu(WA2,JA)|_])
    -> tulisBaris(Stream, 'kartu_aksi_terakhir', PemainAksi-(WA2-JA))
    ; tulisBaris(Stream, 'kartu_aksi_terakhir', tidak_ada)
    ).
```

State yang disimpan meliputi: urutan pemain, giliran aktif, kartu teratas discard pile, kartu tiap pemain, arah permainan, warna aktif, status UNI, sisa deck aktif, serta kartu aksi terakhir. Khusus kartu_aksi_terakhir, digunakan struktur (->): bila riwayatAksi/1 memuat aksi terakhir, disimpan dalam bentuk Pemain-(Warna-Jenis); bila kosong, disimpan sebagai tidak_ada. Informasi ini diperlukan agar fitur mimic danantang dapat dipulihkan dengan benar.

Helper tulisBaris/3 menulis satu baris dengan format Key:Value diikuti baris baru.

```
tulisBaris(Stream, Key, Value) :-
    write(Stream, Key),
    write(Stream, ':'),
    writeq(Stream, Value),
    nl(Stream).
```

Untuk kartu di tangan pemain, representasi internal kartu(Warna, Jenis) dikonversi menjadi bentuk Warna-Jenis agar sesuai format berkas. Konversi ini memanfaatkan operator - bawaan Prolog sehingga write/2 mencetak merah-5 dari pada kartu(merah,5).

```
konvKartu(kartu(W,J), W-J).

konvListKartu([], []).
konvListKartu([K|TK], [P|TP]) :-
    konvKartu(K, P),
```

```

konvListKartu(TK, TP).

tulisKartuPemain(Stream, P) :-
    kartuPemain(P, ListKartu),
    konvListKartu(ListKartu, ListPair),
    atom_concat('kartu_', P, Key),
    tulisBaris(Stream, Key, ListPair).

tulisSemuaKartu(_, []).
tulisSemuaKartu(Stream, [P|Sisa]) :-
    tulisKartuPemain(Stream, P),
    tulisSemuaKartu(Stream, Sisa).

```

Predikat `tulisSemuaKartu/2` melakukan rekursi terhadap list pemain dan menuliskan satu baris `kartu_NamaPemain:[...]` untuk setiap pemain. Nama kunci dibentuk dinamis melalui `atom_concat('kartu_', NamaPemain, Key)` sehingga bekerja untuk nama pemain apa pun

4.4.2 loadGame

Predikat `loadGame` memuat kembali kondisi permainan dari berkas `.txt` yang sebelumnya disimpan oleh `saveGame`. Program meminta nama berkas; jika berkas ditemukan, seluruh state lama dihapus lalu diganti dengan state dari berkas. Jika berkas tidak ada, ditampilkan pesan kesalahan.

```

tulisBaris(Stream, Key, Value) :-
    write(Stream, Key),
    write(Stream, ':'),
    writeq(Stream, Value),
    nl(Stream).

```

Penanganan keberadaan berkas dilakukan dengan `catch/3` yang membungkus `open/3`. Bila berkas tidak ada, `open/3` melempar exception yang ditangkap `catch/3` lalu dialihkan menjadi fail, sehingga cabang `else` menampilkan pesan error. Bila berkas ada, state lama dibersihkan dengan `bersihkanState`, isi berkas dibaca baris demi baris, lalu ditampilkan konfirmasi beserta giliran yang dilanjutkan.

```

bersihkanState :-
    retractall(pemain(_)),
    retractall(giliran(_)),
    retractall(kartuPemain(_, _)),
    retractall(discardTop(_)),
    retractall(arahPermainan(_)),
    retractall(warnaAktif(_)),
    retractall(statusUNI(_)),
    retractall(deckAktif(_)),
    retractall(riwayatAksi(_)).

```

Predikat `bacaSemuaBaris/1` melakukan loop rekursif membaca setiap baris hingga akhir berkas (`end_of_file`), melewati baris kosong, dan memproses baris berisi data.

```

bacaSemuaBaris(Stream) :-

```

```

    bacaLine(Stream, Line),
    ( Line = end_of_file
    -> true
    ; Line = []
    -> bacaSemuaBaris(Stream)
    ; prosesLine(Line),
      bacaSemuaBaris(Stream)
    ).

```

Predikat `bacaLine/2` membaca satu baris karakter per karakter menggunakan `get_char/2` hingga menemui newline atau akhir berkas. Karakter `\r` ditangani secara khusus agar berkas dengan akhir baris gaya Windows (CRLF) tetap terbaca benar.

```

bacaLine(Stream, Result) :-
    get_char(Stream, C),
    bacaLineLanjut(Stream, C, Result).

bacaLineLanjut(_, end_of_file, end_of_file) :- !.
bacaLineLanjut(Stream, '\r', []) :- !,
    get_char(Stream, _).
bacaLineLanjut(_, '\n', []) :- !.
bacaLineLanjut(Stream, C, [C|Rest]) :-
    bacaLine(Stream, Hasil),
    ( Hasil = end_of_file
    -> Rest = []
    ; Rest = Hasil
    ).

```

Setiap baris diproses oleh `prosesLine/1`. Baris dipecah pada karakter : pertama menjadi bagian key dan value. Pemecahan manual ini diperlukan karena prioritas operator : (200) dan - (500) di Prolog tidak kompatibel bila baris seperti `discard_top:merah-6` diparsing utuh. Bagian value kemudian diparsing menjadi term Prolog menggunakan `open_input_chars_stream/2` dan `read/2`.

```

prosesLine(Chars) :-
    splitDiTitikDua(Chars, KeyChars, ValueChars),
    atom_chars(Key, KeyChars),
    appends(ValueChars, ['.', ' '], ValueLengkap),
    open_input_chars_stream(ValueLengkap, S),
    read(S, Value),
    close_input_chars_stream(S),
    handleKV(Key, Value).

splitDiTitikDua([': '|Rest], [], Rest) :- !.
splitDiTitikDua([C|Rest], [C|KeyRest], Value) :-
    splitDiTitikDua(Rest, KeyRest, Value).

```

Predikat `handleKV/2` mendistribusikan setiap pasangan key-value ke fakta yang sesuai menggunakan `assertz/1`. Klausula khusus menangani `deck_aktif` (disimpan apa adanya sebagai list kartu) dan `kartu_aksi_terakhir` (dipulihkan ke `riwayatAksi/1`, baik kosong maupun berisi satu aksi terakhir). Klausula terakhir menangani kunci `kartu>NamaPemain` secara dinamis dengan mengekstrak nama pemain melalui `atom_concat/3`.

```

handleKV(urutan_pemain, ListPemain) :- !,
    assertz(pemain(ListPemain)).
handleKV(giliran, Pemain) :- !,
    assertz(giliran(Pemain)).
handleKV(discard_top, W-J) :- !,
    assertz(discardTop(kartu(W,J))).
handleKV(arah_permainan, Arah) :- !,
    assertz(arahPermainan(Arah)).
handleKV(warna_aktif, Warna) :- !,
    assertz(warnaAktif(Warna)).
handleKV(status_UNI, List) :- !,
    assertz(statusUNI(List)).
handleKV(deck_aktif, Deck) :- !,
    assertz(deckAktif(Deck)).
handleKV(kartu_aksi_terakhir, tidak_ada) :- !,
    assertz(riwayatAksi([])).
handleKV(kartu_aksi_terakhir, Pemain-(W-J)) :- !,
    assertz(riwayatAksi([Pemain-kartu(W,J)])).
handleKV(Key, ListPair) :-
    atom_concat('kartu_', Pemain, Key), !,
    konvListPairToKartu(ListPair, ListKartu),
    assertz(kartuPemain(Pemain, ListKartu)).

```

Bentuk Warna-Jenis pada berkas dikonversi kembali menjadi kartu(Warna, Jenis) melalui konvListPairToKartu/2 sebelum di-assert sebagai fakta kartuPemain/2.

```

konvPairToKartu(W-J, kartu(W,J)).

konvListPairToKartu([], []).
konvListPairToKartu([P|TP], [K|TK]) :-
    konvPairToKartu(P, K),
    konvListPairToKartu(TP, TK).

```

4.5 Deck

4.5.1 deck

```

deck([
    kartu(merah,0),
    kartu(merah,1),
    kartu(merah,2),
    kartu(merah,3),
    kartu(merah,4),
    kartu(merah,5),
    kartu(merah,6),
    kartu(merah,7),
    kartu(merah,8),
    kartu(merah,9),
    kartu(merah,draw_two),
    kartu(merah,skip),
    kartu(merah,reverse),

    kartu(biru,0),
    kartu(biru,1),
    kartu(biru,2),
    kartu(biru,3),
    kartu(biru,4),
    kartu(biru,5),
    kartu(biru,6),
    kartu(biru,7),

```

```

kartu(biru,8),
kartu(biru,9),
kartu(biru,draw_two),
kartu(biru,skip),
kartu(biru,reverse),

kartu(hijau,0),
kartu(hijau,1),
kartu(hijau,2),
kartu(hijau,3),
kartu(hijau,4),
kartu(hijau,5),
kartu(hijau,6),
kartu(hijau,7),
kartu(hijau,8),
kartu(hijau,9),
kartu(hijau,draw_two),
kartu(hijau,skip),
kartu(hijau,reverse),

kartu(kuning,0),
kartu(kuning,1),
kartu(kuning,2),
kartu(kuning,3),
kartu(kuning,4),
kartu(kuning,5),
kartu(kuning,6),
kartu(kuning,7),
kartu(kuning,8),
kartu(kuning,9),
kartu(kuning,draw_two),
kartu(kuning,skip),
kartu(kuning,reverse),

kartu(hitam,wild),
kartu(hitam,wild_draw_four),
kartu(hitam,mimic)

```

1).

Rule ini merepresentasikan kumpulan seluruh kartu dalam permainan sebelum dibagikan.

Isi deck terdiri dari:

- kartu angka (0–9) untuk tiap warna (merah, biru, hijau, kuning)
- kartu aksi:
 - draw_two
 - skip
 - reverse
- kartu khusus:
 - wild
 - wild_draw_four
 - mimic

Fungsi dari rule ini adalah menjadi sumber utama kartu, digunakan saat distribusi kartu awal, dan digunakan untuk pengambilan kartu secara acak.

4.5.2 Random dan Pembagian Kartu

4.5.2.1 ambilIndex

```

ambilIndex(0, [H|T], H, T).
ambilIndex(N, [H|T], X, [H|Sisa]) :-

```

```
N > 0,  
N1 is N - 1,  
ambilIndex(N1, T, X, Sisa).
```

Rule ini digunakan untuk mengambil elemen pada indeks tertentu dari list.

Cara kerja:

- jika index = 0 → ambil head
- jika index > 0 → rekursif ke tail

Fungsi rule ini adalah membantu proses pengambilan kartu acak dari deck.

4.5.2.2 panjang

```
panjang([], 0).  
panjang([_|T], Hasil) :-  
    panjang(T, Sisa),  
    Hasil is Sisa + 1.
```

Rule ini menghitung jumlah elemen dalam list secara rekursif.

Cara kerja:

- list kosong → panjang = 0
- jika ada head → +1 dari tail

Fungsi rule ini adalah digunakan untuk menentukan batas random index dan dasar validasi ukuran deck

4.5.2.3 ambilElemen

```
ambilElemen(List, Elemen, Sisa) :-  
    panjang(List, Panjang),  
    random(0, Panjang, Index),  
    ambilIndex(Index, List, Elemen, Sisa).
```

Rule ini mengambil satu elemen secara acak dari list.

Cara kerja:

- hitung panjang list
- generate index random
- ambil elemen di index tersebut
- return elemen + sisa list tanpa elemen itu

Fungsi rule ini adalah mekanisme utama pengambilan kartu random dari deck

4.5.2.4 bagiKartu

```
bagiKartu(_, Deck, Deck, 0).  
bagiKartu(Pemain, DeckAwal, DeckAkhir, N) :-  
    N > 0,  
    ambilElemen(DeckAwal, Kartu, SisaDeck),  
    (  
        kartuPemain(Pemain, ListLama)  
        ->  
        retract(kartuPemain(Pemain, ListLama)),  
        ListBaru = [Kartu|ListLama]  
        ;  
        ListBaru = [Kartu]  
    ),  
    assertz(kartuPemain(Pemain, ListBaru)),
```



```
N1 is N - 1,  
bagiKartu(Pemain, SisaDeck, DeckAkhir, N1).
```

Rule ini digunakan untuk membagikan N kartu ke satu pemain.

Cara kerja:

- hitung panjang list
- generate index random
- ambil elemen di index tersebut
- return elemen + sisa list tanpa elemen itu

Fungsi rule ini adalah distribusi kartu awal (biasanya 7 kartu per pemain).

4.5.2.5 bagiSemua

```
bagiSemua([], Deck, Deck).  
bagiSemua([P|Tail], DeckAwal, DeckAkhir) :-  
    bagiKartu(P, DeckAwal, DeckBaru, 7),  
    bagiSemua(Tail, DeckBaru, DeckAkhir).
```

Rule ini digunakan untuk membagikan kartu ke semua pemain secara bergiliran.

Cara kerja:

- hitung panjang list
- generate index random
- ambil elemen di index tersebut
- return elemen + sisa list tanpa elemen itu

Fungsi rule ini adalah setup awal permainan.

4.5.3 initDiscard

```
initDiscard(DeckAwal, DeckAkhir) :-  
    ambilElemen(DeckAwal, Kartu, SisaDeck),  
    (  
        Kartu = kartu(Warna, Angka),  
        angkaKartu(Angka)  
        ->  
        assertz(discardTop(Kartu)),  
        assertz(warnaAktif(Warna)),  
        DeckAkhir = SisaDeck  
        ;  
        initDiscard(SisaDeck, DeckAkhir)  
    ).
```

Rule ini menentukan kartu awal di discard pile (kartu tengah permainan).

Cara kerja:

- ambil kartu dari deck secara acak
- cek apakah kartu angka (0–9)
 - jika YA → valid sebagai kartu awal
 - jika TIDAK → ulang ambil kartu

Fungsi rule ini adalah menentukan kartu awal permainan dan memastikan game dimulai dengan kartu angka (bukan kartu aksi).

4.5.4 angkaKartu

```
angkaKartu(0).  
angkaKartu(1).
```

```

angkaKartu(2).
angkaKartu(3).
angkaKartu(4).
angkaKartu(5).
angkaKartu(6).
angkaKartu(7).
angkaKartu(8).
angkaKartu(9).

```

Rule ini menyatakan bahwa nilai tersebut adalah kartu angka valid.

Fungsi rule ini adalah validasi kartu untuk `initDiscard` dan membedakan kartu angka vs kartu aksi.

4.5.5 *ambilKartuKe*

```

ambilKartuKe(1, [H|T], H, T).
ambilKartuKe(N, [H|T], Kartu, [H|Sisa]) :-
    N > 1,
    N1 is N - 1,
    ambilKartuKe(N1, T, Kartu, Sisa).

```

Rule ini mengambil elemen ke-N dari list pemain.

Cara kerja:

- jika $N = 1 \rightarrow$ ambil head
- jika $N > 1 \rightarrow$ rekursif

Fungsi rule ini adalah dipakai saat pemain memilih kartu dari hand.

4.5.6 *kartuValid*

```

kartuValid(kartu(_,draw_two), kartu(_,draw_two)) :- !, fail.
kartuValid(kartu(Warna,draw_two), kartu(Warna,_)) :- !.
kartuValid(kartu(hitam,wild), kartu(hitam,wild)) :- !, fail.
kartuValid(kartu(hitam,wild), _) :- !.
kartuValid(kartu(hitam,wild_draw_four), kartu(hitam,wild_draw_four)) :- !,
fail.
kartuValid(kartu(hitam,wild_draw_four), _) :- !.
kartuValid(kartu(hitam,mimic), kartu(hitam,mimic)) :- !, fail.
kartuValid(kartu(hitam,mimic), _) :- !.
kartuValid(kartu(Warna,_), _) :-
    warnaAktif(Warna).
kartuValid(kartu(_,Jenis), kartu(_,Jenis)).

```

Rule ini menentukan apakah kartu yang dimainkan valid terhadap kartu di discard pile.

Logika validasi:

- `draw_two` tidak boleh stacking sama
- `wild` selalu valid
- `wild_draw_four` selalu valid
- `mimic` selalu valid
- kartu valid jika:
 - warna sama dengan warna aktif
 - atau jenis sama dengan kartu top

Fungsi rule ini adalah inti aturan permainan UNI.

4.5.7 *tampilkanSatuKartu*

```

tampilkanSatuKartu(kartu(Warna,Jenis)) :-
    write(Warna),

```

```

write('-'),
write(Jenis),
nl.

/* Output Format */
/* merah-5 */
/* biru-skip */

```

Rule ini menentukan apakah kartu yang dimainkan valid terhadap kartu di discard pile. Fungsi rule ini adalah debugging dan UI terminal.

4.5.8 updateWarnaAktif

```

updateWarnaAktif(kartu(hitam,_)).
updateWarnaAktif(kartu(Warna,_)) :-
    Warna \= hitam,
    retract(warnaAktif(_)),
    assertz(warnaAktif(Warna)).

```

Rule ini mengatur warna aktif permainan.

Logika:

- jika kartu hitam → tidak mengubah warna
- jika kartu berwarna → update warna aktif

Fungsi rule ini adalah menjaga status warna dalam permainan.

4.5.9 nilaiKartu

```

updateWarnaAktif(kartu(hitam,_)).
updateWarnaAktif(kartu(Warna,_)) :-
    Warna \= hitam,
    retract(warnaAktif(_)),
    assertz(warnaAktif(Warna)).

```

Rule ini memberikan skor untuk setiap jenis kartu.

Mapping:

- angka → sesuai nilai
- skip/reverse/draw_two → 10
- wild/wild_draw_four/mimic → 20

Fungsi rule ini adalah scoring system (ranking atau evaluasi akhir).

4.6 Command

Predikat lihatCommand menampilkan daftar aksi yang dapat dilakukan pemain pada giliran tersebut, terbagi menjadi aksi utama dan aksi pendukung. Daftar aksi utama bersifat kondisional bergantung pada fakta efekPending/1, sedangkan aksi pendukung selalu tetap.

```

lihatCommand :-
    write('Aksi utama yang tersedia:'), nl,
    aksiUtama(ListUtama),
    cetakBernomor(ListUtama, 1),
    nl,
    write('Aksi pendukung yang tersedia:'), nl,
    aksiPendukung(ListPendukung),

```

```
cetakBernomor(ListPendukung, 1).
```

Predikat aksiUtama/1 menentukan daftar aksi utama berdasarkan efek yang sedang menunggu. Jika efek terakhir adalah draw_four, pilihan terbatas pada ambilKartu danantang. Jika draw_two, hanya ambilKartu. Pada kondisi normal, tersedia mainkanKartu, ambilKartu, dan uni.

```
aksiUtama([ambilKartu, tantang]) :-  
    efekPending(draw_four), !.  
aksiUtama([ambilKartu]) :-  
    efekPending(draw_two), !.  
aksiUtama([mainkanKartu, ambilKartu, uni]).
```

Cut (!) pada dua klausa pertama memastikan pemilihan bersifat deterministik: begitu satu kondisi efekPending terpenuhi, Prolog tidak mencoba klausa berikutnya. Jika tidak ada efek yang cocok, klausa ketiga (default) yang dipakai.

Daftar aksi pendukung bersifat tetap dan tidak bergantung pada kondisi permainan.

```
aksiPendukung([lihatCommand, lihatKartu, cekInfo]).
```

Helper cetakBernomor/2 mencetak setiap elemen list dengan nomor urut secara rekursif.

```
cetakBernomor([], _).  
cetakBernomor([H|T], N) :-  
    write(N), write(' '), write(H), nl,  
    N1 is N + 1,  
    cetakBernomor(T, N1).
```

Nilai efekPending/1 diubah melalui helper setEfekPending/1 yang dipanggil oleh bagian gameplay ketika kartu draw_two atau draw_four dimainkan, maupun ketika efek tersebut telah diselesaikan.

```
setEfekPending(Efek) :-  
    retractall(efekPending(_)),  
    assertz(efekPending(Efek)).
```

5 Alur Permainan

5.1 Awal Permainan

```
| ?- startGame.  
Masukkan jumlah pemain: 2.  
Masukkan nama pemain: 'A'.  
Masukkan nama pemain: 'B'.  
Game berhasil dimulai.  
Urutan permainanya adalah:[A,B]  
Kartu Top pertama: kartu(merah,0)  
Giliran A  
  
(1 ms) yes
```

- **Kasus ketika jumlah player tidak sesuai**

```
| ?- startGame.  
Masukkan jumlah pemain: 5.  
Mohon masukkan angka antara 2 - 4.  
Masukkan jumlah pemain:
```

- **Kasus ketika Nama sudah dipakai**

```
| ?- startGame.  
Masukkan jumlah pemain: 2.  
Masukkan nama pemain: 'A'.  
Masukkan nama pemain: 'A'.  
Nama Sudah Digunakan. Masukkan nama lain: 'B'.  
Game berhasil dimulai.  
Urutan permainanya adalah:[A,B]  
Kartu Top pertama: kartu(biru,1)  
Giliran A  
  
(1 ms) yes
```

5.2 Support Command

- **cekInfo**

```
| ?- cekInfo.  
Kartu discard top: merah-5  
Urutan pemain: [Joko,Owo,Lil]  
Arah permainan: kanan  
Giliran: Joko  
  
Nama pemain 1: Joko  
Jumlah kartu: 3  
  
Nama pemain 2: Owo  
Jumlah kartu: 2  
  
Nama pemain 3:Lil  
Jumlah kartu: 2  
  
yes
```

- **lihatKartu**

```
| ?- lihatKartu, !.  
Berikut kartu yang anda miliki.  
1. merah-7  
2. biru-1  
3. hijau-skip  
  
yes
```

- **lihatCommand**

```
| ?- lihatCommand, !.  
Aksi utama yang tersedia:  
1. mainkanKartu  
2. ambilKartu  
3. uni  
  
Aksi pendukung yang tersedia:  
1. lihatCommand  
2. lihatKartu  
3. cekInfo  
  
yes
```

5.3 Validitas Kartu

- **mainkanKartu (warna sama)**

```
| ?- cekInfo.  
Kartu discard top: merah-4  
Urutan pemain: [Owo,Lil,Joko]  
Arah permainan: kanan  
Giliran: Owo  
  
Nama pemain 1: Owo  
Jumlah kartu: 7  
  
Nama pemain 2: Lil  
Jumlah kartu: 7  
  
Nama pemain 3: Joko  
Jumlah kartu: 7  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hijau-8  
2. merah-7  
3. merah-9  
4. biru-1  
5. biru-reverse  
6. hijau-4  
7. biru-9  
  
yes  
| ?- mainkanKartu(2).  
Owo memainkan kartu: merah-7  
  
true ?  
  
yes
```

- ***mainkanKartu (angka sama)***

```
| ?- cekInfo.  
Kartu discard top: merah-7  
Urutan pemain: [Owo,Lil,Joko]  
Arah permainan: kanan  
Giliran: Joko  
  
Nama pemain 1: Owo  
Jumlah kartu: 6  
  
Nama pemain 2: Lil  
Jumlah kartu: 8  
  
Nama pemain 3: Joko  
Jumlah kartu: 7  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. merah-5  
2. kuning-0  
3. merah-skip  
4. merah-8  
5. kuning-7  
6. hijau-9  
7. kuning-draw_two  
  
(1 ms) yes  
| ?- mainkanKartu(5).  
Joko memainkan kartu: kuning-7  
  
true ?  
  
yes
```

5.4 Kartu Aksi

- ***Kartu skip***

```
| ?- cekInfo.  
Kartu discard top: merah-0  
Urutan pemain: [Joko,Owo]  
Arah permainan: kanan  
Giliran: Joko  
  
Nama pemain 1: Joko  
Jumlah kartu: 7  
  
Nama pemain 2: Owo  
Jumlah kartu: 7  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hijau-reverse
```

```

2. hijau-skip
3. merah-2
4. merah-skip
5. hijau-3
6. biru-skip
7. kuning-2

yes
| ?- mainkanKartu(4).
Joko memainkan kartu: merah-skip
Pemain Owo telah di-skip

true ?

yes
| ?- cekInfo.
Kartu discard top: merah-skip
Urutan pemain: [Joko,Owo]
Arah permainan: kanan
Giliran: Joko

Nama pemain 1: Joko
Jumlah kartu: 6

Nama pemain 2: Owo
Jumlah kartu: 7

yes

```

● **Kartu reverse**

```

| ?- cekInfo.
Kartu discard top: hijau-skip
Urutan pemain: [Joko,Owo]
Arah permainan: kanan
Giliran: Joko

Nama pemain 1: Joko
Jumlah kartu: 5

Nama pemain 2: Owo
Jumlah kartu: 7

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. hijau-reverse
2. merah-2
3. hijau-3
4. biru-skip
5. kuning-2

yes
| ?- mainkanKartu(1).
Joko memainkan kartu: hijau-reverse
KARTU REVERSE DIMAINKAN!
Arah permainan dibalik!

```



```
true ?

yes
| ?- cekInfo.
Kartu discard top: hijau-reverse
Urutan pemain: [Joko,Owo]
Arah permainan: kiri
Giliran: Owo

Nama pemain 1: Joko
Jumlah kartu: 4

Nama pemain 2: Owo
Jumlah kartu: 7

yes
```

- **Kartu draw two**
 - **Kasus berhasil**

```
| ?- cekInfo.
Kartu discard top: kuning-1
Urutan pemain: [Joko,Owo]
Arah permainan: kanan
Giliran: Owo

Nama pemain 1: Joko
Jumlah kartu: 8

Nama pemain 2: Owo
Jumlah kartu: 7

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-1
2. merah-reverse
3. kuning-2
4. kuning-draw_two
5. kuning-0
6. biru-7
7. hijau-9

yes
| ?- mainkanKartu(4).
Owo memainkan kartu: kuning-draw_two
Joko mendapatkan 2 kartu dan kehilangan giliran!

true ?

yes
| ?- cekInfo.
Kartu discard top: kuning-draw_two
Urutan pemain: [Joko,Owo]
Arah permainan: kanan
```

```
Giliran: Owo

Nama pemain 1: Joko
Jumlah kartu: 10

Nama pemain 2: Owo
Jumlah kartu: 6

yes
```

- ***Kasus draw two ditumpuk***

```
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. biru-0
2. kuning-3
3. kuning-skip
4. kuning-0
5. biru-draw_two
6. hijau-draw_two
7. merah-5

yes
| ?- mainkanKartu(6).
Owo memainkan kartu: hijau-draw_two
Joko mendapatkan 2 kartu dan kehilangan giliran!

true ?

Yes
/* karna playernya hanaya dua jadi balik ke diri sendiri */
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. biru-0
2. kuning-3
3. kuning-skip
4. kuning-0
5. biru-draw_two
6. merah-5

yes
| ?- mainkanKartu(5).

no
```

- ***Kartu wild dan wild draw four***

```
| ?- cekInfo.
Kartu discard top: kuning-draw_two
Urutan pemain: [Joko,Owo]
Arah permainan: kanan
Giliran: Joko

Nama pemain 1: Joko
Jumlah kartu: 12
```

Nama pemain 2: Owo

Jumlah kartu: 9

yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. hitam-wild_draw_four

2. biru-4

3. kuning-3

4. hijau-5

5. biru-3

6. merah-4

7. hijau-1

8. kuning-7

9. kuning-reverse

10. hijau-6

11. hijau-reverse

12. biru-9

yes

| ?- mainkanKartu(1).

Joko memainkan kartu: hitam-wild_draw_four

Pilih warna baru(merah, kuning, hijau, biru): biru.

Owo mendapatkan 4 kartu dan kehilangan giliran!

true ?

yes

| ?- cekInfo.

Kartu discard top: hitam-wild_draw_four

Urutan pemain: [Joko,Owo]

Arah permainan: kanan

Giliran: Joko

Nama pemain 1: Joko

Jumlah kartu: 11

Nama pemain 2: Owo

Jumlah kartu: 13

Yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. biru-4

2. kuning-3

3. hijau-5

4. biru-3

5. merah-4

6. hijau-1

7. kuning-7

8. kuning-reverse

9. hijau-6

10. hijau-reverse

11. biru-9

(1 ms) yes

| ?- mainkanKartu(11).

Joko memainkan kartu: biru-9

```
true ?  
yes
```

- **Kartu mimic (bonus)**

```
| ?- mainkanKartu(3).  
  
Joko memainkan kartu merah-skip.  
Giliran Owo dilewati.  
Giliran Lil.  
  
yes  
  
| ?- mainkanKartu(2).  
Pilih warna baru (merah/kuning/hijau/biru): biru.  
  
Lil memainkan kartu hitam-mimic.  
Mimic meniru efek kartu skip.  
Warna aktif sekarang biru.  
Giliran Joko dilewati.  
Giliran Owo.  
  
yes
```

5.5 UNI dan Tangkap

- **UNI tidak valid**

```
| ?- cekInfo.  
Kartu discard top: biru-9  
Urutan pemain: [Joko,Owo]  
Arah permainan: kanan  
Giliran: Owo  
  
Nama pemain 1: Joko  
Jumlah kartu: 10  
  
Nama pemain 2: Owo  
Jumlah kartu: 13  
  
yes  
| ?- uni(1).  
Tidak valid, sisa kartu selanjutnya bukanlah 1!  
Owo mendapatkan penalti 1 kartu acak  
  
true ?  
  
(1 ms) yes
```

- **UNI valid**

```
| ?- uni(1).
Joko memainkan kartu: merah-7
Joko mengucapkan UNI! Kartu sisa 1

yes
| ?- cekInfo.
Kartu discard top: merah-7
Urutan pemain: [Joko,Owo,Lil]
Arah permainan: kanan
Giliran: Owo

Nama pemain 1: Joko
Jumlah kartu: 1

Nama pemain 2: Owo
Jumlah kartu: 2

Nama pemain 3: Lil
Jumlah kartu: 2

yes
```

- **Tangkap valid**

```
| ?- cekInfo.
Kartu discard top: hitam-wild_draw_four
Urutan pemain: [Joko,Owo,Lil]
Arah permainan: kanan
Giliran: Lil

Nama pemain 1: Joko
Jumlah kartu: 1

Nama pemain 2: Owo
Jumlah kartu: 6

Nama pemain 3: Lil
Jumlah kartu: 2

yes
| ?- tantang.
Tantangan berhasil! Owo mendapat 4 kartu acak!

Yes

| ?- cekInfo.
Kartu discard top: hitam-wild_draw_four
Urutan pemain: [Joko,Owo,Lil]
Arah permainan: kanan
Giliran: Lil

Nama pemain 1: Joko
Jumlah kartu: 1

Nama pemain 2: Owo
Jumlah kartu: 10
```

```
Nama pemain 3: Lil  
Jumlah kartu: 2
```

```
yes
```

- **Tangkap tidak valid**

```
| ?- cekInfo.  
Kartu discard top: biru-9  
Urutan pemain: [Joko,Owo]  
Arah permainan: kanan  
Giliran: Joko  
  
Nama pemain 1: Joko  
Jumlah kartu: 10  
  
Nama pemain 2: Owo  
Jumlah kartu: 14  
  
yes  
| ?- tangkap(Joko).  
Kamu salah tangkap! Mendapatkan penalti 1 kartu  
  
true ?  
  
yes
```

5.6 End Game

```
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hijau-4  
  
yes  
| ?- mainkanKartu(1).  
Owo memainkan kartu: hijau-4  
  
game selesai  
Berikut perhitungan poin sisa kartu.  
Joko: hitam-wild + hijau-7 + kuning-1 + biru-draw_two + hijau-3 = 41 poin  
Owo: kartu habis = 0 poin  
  
urutan pemain:  
Owo Joko  
  
true ?  
  
yes
```

5.7 Save load

- **Save (Query)**

```

| ?- startGame
.
Masukkan jumlah pemain: 2.
Masukkan nama pemain: 'Joko'.
Masukkan nama pemain: 'Owo'.
Game berhasil dimulai.
Urutan pemainnya adalah: [Joko,Owo]
Kartu Top pertama: kartu(merah,0)
Giliran Joko

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. hijau-reverse
2. hijau-skip
3. merah-2
4. merah-skip
5. hijau-3
6. biru-skip
7. kuning-2

yes
| ?- mainkanKartu(3).
Joko memainkan kartu: merah-2

true ?

yes
| ?- saveGame.
Masukkan nama file penyimpanan: contohhaje
.
Status permainan berhasil disimpan ke contohhaje.txt.

true ?

yes

```

● **Save (Output file)**

```

urutan_pemain:['Joko','Owo']
giliran:'Owo'
discard_top:merah-2
kartu_Joko:[hijau-reverse,hijau-skip,merah-skip,hijau-3,biru-skip,kuning-2]
kartu_Owo:[hijau-4,merah-4,merah-1,kuning-7,hijau-1,biru-7,kuning-reverse]
arah_permainan:kanan
warna_aktif:merah
status_UNI:[]
deck_aktif:[kartu(merah,3),kartu(merah,5),kartu(merah,6),kartu(merah,7),kartu(
merah,8),kartu(merah,9),kartu(merah,draw_two),kartu(merah,reverse),kartu(biru,
0),kartu(biru,1),kartu(biru,2),kartu(biru,3),kartu(biru,4),kartu(biru,5),kartu
(biru,6),kartu(biru,8),kartu(biru,9),kartu(biru,draw_two),kartu(biru,reverse),
kartu(hijau,0),kartu(hijau,2),kartu(hijau,5),kartu(hijau,6),kartu(hijau,7),kar
tu(hijau,8),kartu(hijau,9),kartu(kuning,0),kartu(kuning,1),kartu(kuning,3),kar
tu(kuning,4),kartu(kuning,5),kartu(kuning,6),kartu(kuning,8),kartu(kuning,9),k
artu(kuning,draw_two),kartu(kuning,skip),kartu(hitam,wild),kartu(hitam,wild_dr
aw_four),kartu(hitam,mimic)]
kartu_aksi_terakhir:tidak_ada

```

● **Load**

```
| ?- loadGame.  
Masukkan nama file yang akan dimuat: contohhaje.  
Status permainan berhasil dimuat dari contohhaje.txt.  
Melanjutkan giliran Owo.  
  
(1 ms) yes
```

5.8 Contoh Full Game

```
| ?- startGame.  
Masukkan jumlah pemain: 2.  
Masukkan nama pemain: 'Joko'.  
Masukkan nama pemain: 'Owo'.  
Game berhasil dimulai.  
Urutan pemainnya adalah: [Owo,Joko]  
Kartu Top pertama: kartu(hijau,5)  
Giliran Owo  
  
(1 ms) yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. biru-0  
2. kuning-3  
3. kuning-skip  
4. kuning-0  
5. biru-draw_two  
6. hijau-draw_two  
7. merah-5  
  
yes  
| ?- mainkanKartu(6).  
Owo memainkan kartu: hijau-draw_two  
Joko mendapatkan 2 kartu dan kehilangan giliran!  
  
true ?  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. biru-0  
2. kuning-3  
3. kuning-skip  
4. kuning-0  
5. biru-draw_two  
6. merah-5  
  
yes  
| ?- ambilKartu.  
Kamu ngambil kartu dari deck kartu!  
  
true ?  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hijau-4
```



```
2. merah-3
3. biru-6
4. hitam-mimic
5. kuning-4
6. hijau-7
7. biru-5
8. kuning-1
9. merah-2

yes
| ?- mainkanKartu(1) .
Joko memainkan kartu: hijau-4

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. kuning-9
2. biru-0
3. kuning-3
4. kuning-skip
5. kuning-0
6. biru-draw_two
7. merah-5

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-3
2. biru-6
3. hitam-mimic
4. kuning-4
5. hijau-7
6. biru-5
7. kuning-1
8. merah-2

(1 ms) yes
| ?- cekInfo.
Kartu discard top: hijau-4
Urutan pemain: [Owo,Joko]
Arah permainan: kanan
Giliran: Joko

Nama pemain 1: Owo
Jumlah kartu: 8

Nama pemain 2: Joko
Jumlah kartu: 8

yes
| ?- mainkanKartu(4) .
Joko memainkan kartu: kuning-4
```

```
true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-skip
2. kuning-9
3. biru-0
4. kuning-3
5. kuning-skip
6. kuning-0
7. biru-draw_two
8. merah-5

yes
| ?- mainkanKartu(5).
Owo memainkan kartu: kuning-skip
Pemain Joko telah di-skip

true ?

yes
| ?- mainkanKartu(4).
Owo memainkan kartu: kuning-3

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-skip
2. kuning-9
3. biru-0
4. kuning-0
5. biru-draw_two
6. merah-5

yes
| ?- mainkanKartu(2).
Owo memainkan kartu: kuning-9

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-skip
2. biru-0
3. kuning-0
4. biru-draw_two
```

```
5. merah-5

yes
| ?- mainkanKartu(3).
Owo memainkan kartu: kuning-0

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- mainkanKartu(2).
Owo memainkan kartu: biru-0

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-skip
2. biru-draw_two
3. merah-5

(1 ms) yes
| ?- mainkanKartu(2).
Owo memainkan kartu: biru-draw_two
Joko mendapatkan 2 kartu dan kehilangan giliran!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-skip
2. merah-5

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. biru-4
2. biru-1
3. merah-9
4. biru-skip
5. kuning-2
6. biru-8
7. merah-3
```

```
8. biru-6
9. hitam-mimic
10. hijau-7
11. biru-5
12. kuning-1
13. merah-2

yes
| ?- mainkanKartu(4).
Joko memainkan kartu: biru-skip
Pemain Owo telah di-skip

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-7
2. merah-skip
3. merah-5

(1 ms) yes
| ?- mainkanKartu(2).
Owo memainkan kartu: merah-skip
Pemain Joko telah di-skip

true ?

yes
| ?- mainkanKartu(1).
Owo memainkan kartu: merah-7

true ?

yes
| ?- ambilKartu.
Kamu ngambil kartu dari deck kartu!

true ?

yes
| ?- mainkanKartu(1).
Owo memainkan kartu: merah-5

game selesai
Berikut perhitungan poin sisa kartu.
Owo: kartu habis = 0 poin
Joko: kuning-draw_two + hijau-1 + biru-4 + biru-1 + merah-9 + kuning-2 + biru-8
+ merah-3 + biru-6 + hitam-mimic + hijau-7 + biru-5 + kuning-1 + merah-2 = 79
poin

urutan pemain:
Owo Joko

true ?
```

yes

6 Pembagian Kerja dalam Kelompok

Nama	Pembagian kerja	Persentase
Mochamad Fachri Alfaridzi	Cek validasi, Exit condition, Simpan sisa kartu ke deckAktif, Validasi jumlah pemain,random urutan main	100%
Zidane Uland Fakhry	lihatKartu, cekInfo, Implementasi gameplay dasar, Randomisasi deck.	100%
Moch Naufal Zaki Basara	Uni dan Tangkap, efek kartu skip, reverse, wild, draw,antang, ambil kartu random.	100%
Haikal Muhammad Royyan	saveGame, loadGame, lihatCommand.	100%